

DETAILED COMPREHENSIVE ANSWERS - PART 2

AES-128, Verification Methodology, and Security Concepts

AES-128 ENCRYPTION CORE - DETAILED ARCHITECTURE

Describe your AES-128 encryption core architecture and design philosophy in detail

AES-128 is the symmetric encryption standard specified in FIPS 197. My implementation focuses on a clean, verifiable architecture that balances functionality, performance, and testability.

****Overall Architecture****:

The design uses an iterative architecture—rather than unrolling all 11 rounds spatially (which would be huge), I process all 11 rounds sequentially in a state machine. The plaintext flows through 11 iterations, each applying one round of AES transformation.

****Key Components****:

1. ****S-box (Substitution Box)****: This is the critical nonlinear component of AES. FIPS 197 specifies the S-box as a 256-entry lookup table mapping 8-bit inputs to 8-bit outputs. I implemented it using the Boyar-Peralta optimization, which minimizes the gate count using composite field arithmetic and gate minimization algorithms. Instead of a naive 256×8 lookup ROM (which would be large), Boyar-Peralta reduces it to approximately 116 gates while maintaining the correct 256-entry mapping. This was a deliberate choice—I researched different S-box implementations and selected Boyar-Peralta because it's widely used in production, well-documented, and offers the best gate-count-to-functionality trade-off.

2. ****Key Schedule****: Generates 11 round keys from the 128-bit master key. The algorithm:

- Takes the 128-bit input key
- Splits it into four 32-bit words
- Generates 40 additional 32-bit words ($10 \text{ rounds} \times 4 \text{ words per round}$)
- Each new word is derived from the previous word and the word 4 positions back
- Every 4th word involves SubWord (apply S-box), RotWord (rotate bytes), and XOR with round constant (Rcon)

The Rcon (round constant) values are powers of 2 in $GF(2^8)$: 01, 02, 04, 08, 10, 20, 40, 80, 1B, 36 for rounds 1-10.

3. ****State Matrix****: AES operates on a 4×4 matrix of bytes. My implementation represents

this as 16 bytes, with explicit mapping between the linear byte array and the 4×4 matrix when needed.

4. **Round Operations** (applied 11 times, with final round omitting MixColumns):

- **SubBytes**: Each byte in the state passes through the S-box. This is the only nonlinear operation in AES and provides confusion.
- **ShiftRows**: A fixed permutation. Row 0 shifts 0 positions, row 1 shifts 1 position left, row 2 shifts 2, row 3 shifts 3. This provides diffusion across columns.
- **MixColumns** (except final round): A matrix multiplication in $GF(2^8)$. Each column of the state is multiplied by a fixed polynomial. This is where the real diffusion happens—a change in one input byte affects all four output bytes of a column.
- **AddRoundKey**: XOR the state with the round key.

Design Choices:

- **Iterative vs. Unrolled**: I chose iterative to keep area small. Unrolled would be faster but would require 11 copies of round logic.
- **Combinational vs. Pipelined**: Mine is fully combinational—no pipelining across rounds. The latency is ~11 cycles (one round per cycle through the state machine).
- **S-box Implementation**: Boyar-Peralta optimization, as discussed.
- **Interface**: Simple handshake with data_v_i (data valid input), res_v_o (result valid output), stable plaintext and key inputs.

Performance Characteristics:

- Latency: ~11 cycles from asserting data_v_i to asserting res_v_o
- Throughput: One encryption per 11 cycles = approximately 91 megabits/second at 100 MHz
- Area: Moderate (dominated by S-box instances—11 per round, ~120 instances total)
- Power: Primarily dynamic switching power from S-box operations and round key XORs

UVM TESTBENCH ARCHITECTURE - DETAILED IMPLEMENTATION

Describe your complete UVM testbench architecture for AES-128 with specific implementation details

The UVM testbench is a complete verification environment for AES-128, following industry-standard methodology. Let me walk through each component:

Interface (aes_if):

Defines the DUT signals and timing:

- logic clk, nreset
- logic data_v_i (input valid signal)
- logic [127:0] data_i (plaintext)

- logic [127:0] key_i (encryption key)
- logic res_v_o (output valid signal)
- logic [127:0] res_o (ciphertext)

The interface includes clocking blocks for driver and monitor, enabling synchronous stimulus and observation.

****Transaction (aes_item extends uvm_sequence_item)**:**

Contains all relevant data for one AES encryption:

- bit [127:0] data: plaintext
- bit [127:0] key: cipher key
- bit [127:0] result: captured ciphertext
- int unsigned latency: cycles from input valid to output valid
- aes_pattern_e data_pattern, key_pattern: categorize the stimulus
- bit [7:0] data_same_byte, key_same_byte: for SAME_BYTE pattern
- int unsigned data_bit_pos, key_bit_pos: for WALKING_ONE/WALKING_ZERO patterns

Pattern enums define constrained-random coverage bins:

- RANDOM: fully random 128-bit values
- ALL_ZERO: all zeros
- ALL_ONE: all ones
- SAME_BYTE: single byte repeated 16 times
- ALT_A5, ALT_5A: alternating bit patterns
- BYTE_RAMP: bytes increment (0x00, 0x01, ..., 0x0F)
- WALKING_ONE, WALKING_ZERO: single-bit or inverted patterns

****Sequencer (aes_sequencer extends uvm_sequencer #(aes_item))**:**

Generates transaction sequences. The base sequencer handles constraint randomization. Derived test classes specify different sequence generation strategies (random, directed, corner-case).

****Driver (aes_driver extends uvm_driver #(aes_item))**:**

Translates transactions to pin-level stimulus:

1. Waits for reset deassertion
2. Gets next transaction from sequencer
3. Drives nreset, data_v_i = 1, data_i, key_i synchronously
4. On next clock, deasserts data_v_i
5. Waits for res_v_o assertion
6. Captures res_o
7. Records the transaction as complete

Timing is critical—data must be stable when data_v_i asserts.

****Monitor (aes_monitor extends uvm_monitor)**:**

Passively observes DUT behavior:

1. Watches for data_v_i assertion
2. Captures data_i, key_i, current cycle
3. Monitors res_v_o
4. When res_v_o asserts, captures res_o and latency
5. Constructs aes_item transaction
6. Broadcasts via analysis port to scoreboard

The monitor never drives—pure observation. This separation allows passive checking of the design.

****Agent (aes_agent extends uvm_agent)**:**

Bundles driver, monitor, sequencer:

- Creates driver (if active mode)
- Creates monitor (always)
- Creates sequencer (if active mode)
- Connects driver to sequencer
- Provides analysis port from monitor to scoreboard

Parameterized for active/passive operation (useful for later extending testbench for real systems).

****Reference Model (in scoreboard)**:**

Implements FIPS 197 AES-128 in SystemVerilog:

- Function aes_128_encrypt(plaintext, key) returns ciphertext
- Implements all rounds, S-box, SubBytes, ShiftRows, MixColumns, AddRoundKey
- S-box values hardcoded (standard FIPS 197 table)
- Galois field operations (multiply by constants in $GF(2^8)$)
- Produces bit-exact output matching the specification

****Scoreboard (aes_scoreboard extends uvm_scoreboard)**:**

Compares DUT outputs to reference model:

1. Receives transactions from monitor
2. Calls reference model with same plaintext/key
3. Compares expected output to DUT output
4. Records pass/fail
5. Collects functional coverage samples
6. Reports mismatches with detailed diagnostics

The scoreboard is the ultimate arbiter of correctness.

****Functional Coverage (aes_coverage)**:**

Collects coverage samples to measure verification completeness:

- Covergroup for plaintext patterns: bins for each pattern type
- Covergroup for key patterns: bins for each pattern type
- Cross-coverage: plaintext_pattern × key_pattern (36 combinations)
- Byte-level coverage: ensure each plaintext byte position exercises all patterns
- Latency coverage: bins for expected cycle counts
- Feature coverage: data_v_i assertions, res_v_o captures

Coverage goals: 95%+ completion indicates sufficient stimulus diversity.

****Environment (aes_environment extends uvm_environment)**:**

Top-level component integrating all infrastructure:

- Instantiates aes_agent
- Instantiates aes_scoreboard
- Instantiates aes_coverage
- Connects monitor analysis port to scoreboard
- Connects monitor to coverage collector
- Provides configuration interface

****Test Hierarchy (aes_test and derivatives)**:**

- aes_base_test: configures environment, runs for default length
- aes_random_test: enables full randomization, biased toward broad coverage
- aes_directed_test: runs specific known-vector sequences from NIST
- aes_corner_case_test: emphasizes edge cases and boundary conditions

****Testbench Integration**:**

Top-level module instantiates:

- DUT (AES-128 core)
- Interface (aes_if)
- Test program (aes_test and derivatives)
- Clock generator
- Reset driver

The entire environment is self-contained and can run on simulators like VCS, Xcelium, or EDA Playground.

****Why This Architecture Matters**:**

- Modularity: Each component has single responsibility, easy to debug
- Reusability: Agent can be used in larger testbenches (SoC integration)
- Scalability: Adding new tests just means creating new test classes
- Automation: Coverage metrics tracked automatically, not manually
- Professionalism: This is production-quality UVM, not student code

VERIFICATION METHODOLOGY - DETAILED CONCEPTS

Explain gate-level simulation (GLS) in detail and why it's critical for ASIC verification

Gate-level simulation is where simulation meets reality. It's the bridge between ideal behavioral RTL and the actual silicon that will be manufactured. Let me explain why it's essential.

****The Problem It Solves**:**

RTL simulation is behavioral and abstract. When you write:

```
...
```

```
assign sum = a + b;
```

```
...
```

The simulator knows conceptually that addition happens with zero delay. But in real hardware, addition takes time. A 32-bit adder path might have a propagation delay of 2-3 nanoseconds. If your clock period is 10 nanoseconds, that's significant timing closure issue.

More subtly, RTL doesn't model glitches, hazards, or metastability. Signal transitions don't happen instantaneously in hardware. When a multiplexer selects between two changing signals, intermediate values might briefly appear (a glitch). In RTL, you wouldn't see this. At gate-level, glitches become visible.

RTL also doesn't model setup/hold timing constraints for flip-flops. If you're sampling a signal at exactly the clock edge, and the signal is still settling, you have a setup violation. RTL simulation doesn't catch this because it assumes clocking happens magically.

****What GLS Does**:**

Gate-level simulation replaces abstract behavioral code with actual gates—AND, OR, XOR, flip-flops, etc. The simulation tool back-annotates timing delays from a Standard Delay Format (SDF) file, which contains actual path delays extracted from synthesis.

So instead of:

```
...
```

```
assign sum = a + b; // happens in zero time
```

```
...
```

GLS has something like:

```
...
```

```
gate_adder #(delay=2.5ns) add_inst(...); // happens in 2.5ns
```

```
...
```

****The GLS Flow**:**

1. **Synthesis**: RTL is synthesized to a gate-level netlist using a standard cell library (e.g., TSMC 28nm). Synthesis tool applies timing constraints and optimization.
2. **SDF Extraction**: The synthesis tool generates Standard Delay Format file containing:
 - Combinational path delays (e.g., "input a to output sum: 2.5ns")
 - Sequential delays (e.g., "setup time, hold time, clock-to-output for flip-flops")
 - Interconnect delays (e.g., wire capacitance converted to delay)
3. **Gate-Level Testbench**: The same testbenches used for RTL are run at gate-level, but the design under test is the gate-level netlist, not RTL.
4. **Timing Back-Annotation**: The simulator loads the SDF file and applies these delays to all signal transitions.
5. **Waveform Comparison**: Waveforms from RTL and gate-level are compared. Any divergence indicates an RTL-to-synthesis issue.

Why I Did GLS for SERV:

My SERV thesis included GLS verification because the processor is timing-critical. A bit-serial processor is sensitive to metastability and timing path violations. When I synthesized SERV:

1. Ran the testbench at RTL (known working)
2. Synthesized with standard library
3. Generated SDF
4. Ran identical testbench at gate-level with SDF back-annotation
5. Compared waveforms

I discovered a subtle issue: a multiplexer chain selecting between different shift directions (left vs. right) had a timing path that exceeded the clock period by 200 ps. The culprit: SDF showed a particular path through the mux chain was pessimistic—synthesis tool estimated delays conservatively. I worked with the synthesis team to adjust timing constraints to loosen that path, re-synthesized, and re-ran GLS. The second time, GLS passed with zero differences.

What This Caught:

Had I shipped SERV without GLS, it might have synthesized and appeared to work in post-synthesis simulation, but on real silicon at higher temperatures or lower voltages (worst-case corners), that timing path might have violated, causing random failures. GLS forced me to find and fix this proactively.

****GLS Challenges**:**

- Speed: GLS is 10-100x slower than RTL simulation because every gate is modeled explicitly
- Debugging: Waveforms are massive because every gate's behavior is visible
- SDF Accuracy: SDF is only as accurate as the synthesis tool and library timing characterization
- Iterative: GLS failures often require RTL changes and re-synthesis

****Sign-Off Criteria**:**

For production ASICs:

- GLS passes 100% (zero differences vs. RTL)
- All timing constraints met (setup, hold, max delay)
- Power analysis from GLS matches estimates
- Coverage metrics consistent RTL→GLS

Without GLS passing, tape-out is not approved.

SECURITY HARDWARE CONCEPTS - DETAILED UNDERSTANDING

Explain side-channel analysis attacks and how hardware design impacts resistance in detail

Side-channel attacks are attacks on cryptographic systems that don't target the algorithm itself but rather observable physical phenomena—power consumption, execution time, electromagnetic emissions, etc.

****Power Analysis (Differential Power Analysis)**:**

The attack: Cryptographic operations consume power. The amount of power consumed depends on which bits are being manipulated. For example:

- Setting a bit to 1 might charge a capacitor (consuming power)
- Keeping a bit at 0 might not charge it (consuming less power)

An attacker with access to power consumption measurements could correlate power traces with known plaintext to recover secret key bits.

Example: AES key schedule. If the power trace shows a spike at a specific moment, and that spike correlates with a particular key bit pattern, the attacker can infer that key bit. Repeated measurements across many encryptions with different plaintexts allow statistical correlation analysis to recover the entire key.

****Hardware Defense Against Power Analysis**:**

Constant-time implementation is critical. Every code path—whether encrypting with a zero bit or a one bit—should consume identical power and take identical time.

In my AES design, I ensured:

1. All rounds execute unconditionally—no branches based on data
2. S-box is combinational—no conditional logic based on input
3. Round count is fixed (always 11)—no early termination

However, this alone isn't sufficient. Even if algorithmic flow is constant-time, transistor-level power leakage (subthreshold leakage, switching power) still correlates with data.

Advanced techniques:

- ****Masking****: Split every secret value into shares. Instead of s , store s_1 and s_2 where $s = s_1 \oplus s_2$. Perform computations on shares separately, then combine. An attacker measuring power sees contributions from masked values, not secrets directly.
- ****Dynamic power scaling****: Variable voltage/frequency to make power independent of data
- ****Decoupling capacitors****: Reduce power supply noise which can leak information

****Timing Side-Channels**:**

Execution time depends on algorithm paths. Examples:

- Loop iterations (square-and-multiply in RSA—bit-by-bit multiplication)
- Cache hits/misses (table lookups in software AES)
- Early termination (some algorithms stop early if conditions met)

Defense: Constant-time algorithms. AES is naturally constant-time (all rounds always execute), but implementers can violate this with poor coding.

****Electromagnetic (EM) Side-Channels**:**

As current flows through transistors, it generates electromagnetic emissions. Different instructions cause different current patterns, which correlate with data. An attacker with an EM probe near the chip can measure emissions and recover information.

Defense: EM shielding, careful circuit layout to minimize EM radiation, and algorithmic defenses (constant-time).

****Fault Injection Attacks**:**

Adversary deliberately induces errors (voltage glitch, clock glitch, laser pulse) to corrupt

computation. For example:

- Glitch during AES final round to skip the last round
- Glitch during a comparison to make unequal values appear equal
- Glitch during key schedule to corrupt round keys

Defense: Error detection (ECC codes, parity), error correction, redundant computation with comparison.

****How I Would Verify Side-Channel Resistance**** (as I'd do at Qualcomm):

1. ****Power Trace Analysis****:

- Simulate AES with various plaintexts and keys
- Measure power consumption cycle-by-cycle
- Compute statistical correlation between power traces and plaintext/key bits
- Expected result: correlation ≈ 0 (no information leakage)

2. ****Timing Analysis****:

- Measure execution cycles for different inputs
- Expected result: all encryptions take identical cycle count

3. ****Formal Verification****:

- Write properties: "Output is not dependent on secret key bits except through encryption function"
- Use formal tool to prove these properties hold
- This is more rigorous than simulation-based checking

4. ****Fault Injection Testing****:

- Inject bit flips at random times/locations in simulation
- Verify either flips are detected (error flag asserted) or output is obviously corrupted
- Use formal methods to prove critical operations are fault-resistant

5. ****Post-Silicon Measurement**** (hardware tape-out):

- Measure actual power consumption on silicon
- Verify against simulation predictions
- Perform real DPA attacks (with known keys and plaintexts) to confirm resistance